

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 1 – Data Interpretation
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Ch.saisri	Reg. No.:	192472367
Section / Batch:	Cse-ai	Date:	

Code of Conduct

I **ch.saisri** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read each data set, table, semantic representation, or scenario carefully before answering.
- Analyze the given information and provide logical, evidence-based interpretations.
- Show all intermediate reasoning, semantic analysis steps, predicate representations, or calculations wherever applicable.
- Answers should be concise, structured, and technically accurate.
- Support your conclusions using the data provided in the question.
- Draw diagrams, semantic networks, parse trees, or logical representations wherever relevant.
- Avoid copying definitions alone; focus on analyzing the given scenario and interpreting the data.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic representations, identify action–entity relationships, detect interpretation errors, and evaluate meaning representation in chatbot systems.	Q1
First-Order Predicate Calculus & Representing Linguistically Relevant Concepts	Construct predicate representations, apply logical inference rules, derive conclusions, and evaluate reasoning in smart manufacturing environments.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze contextual clues, determine correct word senses, evaluate ambiguity resolution, and improve semantic understanding in search systems.	Q3
Syntax-Driven Semantic Analysis	Analyze syntactic structures, assign semantic roles, evaluate parsing accuracy, and improve semantic interpretation in healthcare NLP applications.	Q4

1. Frame Semantics in Banking Customer-Service Systems

A digital banking platform uses semantic frames to understand customer requests. The NLP system identifies the **action, agent, object, and destination** involved in each request.

Customer Query	Generated Semantic Frame
"Transfer ₹20,000 from my savings account to my son's account."	TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)
"Block my debit card immediately."	BLOCK(DebitCard, Customer)
"Send my transaction statement to my email."	SEND(TransactionStatement, Email, Customer)
"Increase my daily withdrawal limit."	MODIFY(WithdrawalLimit, Customer)

Additional transaction logs show:

Query ID	Actual User Intent	System Interpretation
B1	Transfer money to another account	Transfer money
B2	Block debit card	Block debit card
B3	Receive transaction statement by email	Send statement to email
B4	Increase withdrawal limit	Decrease withdrawal limit

Tasks:

1. Analyze the semantic frame of each banking request and identify the relationship between the action and its participants.
2. Identify the query in which the semantic interpretation is incorrect and explain why.
3. Analyze how incorrect identification of semantic roles can lead to an incorrect banking operation.
4. Recommend a frame-based semantic analysis approach to improve the reliability of banking conversational systems.

2. First-Order Predicate Calculus for Smart Agriculture

A smart agricultural system monitors crop fields using sensors and logical rules.

Field Data:

Field	Soil Condition	Irrigation Status	Crop
F1	Dry	Available	Rice
F2	Wet	Available	Wheat
F3	Dry	Unavailable	Maize
F4	Wet	Unavailable	Rice

The system uses the following predicate rules:

- $\text{Dry}(x) \wedge \text{IrrigationAvailable}(x) \rightarrow \text{NeedsWater}(x)$
- $\text{NeedsWater}(x) \rightarrow \text{Irrigate}(x)$
- $\text{Wet}(x) \rightarrow \neg \text{NeedsWater}(x)$
- $\text{IrrigationUnavailable}(x) \rightarrow \neg \text{Irrigate}(x)$
- $\text{Rice}(x) \wedge \text{NeedsWater}(x) \rightarrow \text{PriorityIrrigation}(x)$

Tasks:

1. Represent the field observations using First-Order Predicate Calculus.
2. Using the given rules, determine which fields require irrigation.
3. Determine whether F1 and F3 should be treated differently by the automated irrigation controller.
4. Analyze whether predicate logic alone is sufficient for agricultural decision support when sensor information is incomplete or uncertain.

3. Word Sense Disambiguation in a Travel Recommendation System

A travel platform receives search queries containing ambiguous words. The recommendation engine uses surrounding words and user interactions to determine the intended meaning.

Search Query	Possible Sense 1	Possible Sense 2
"Amazon tour"	Amazon rainforest	Amazon company

Search Query	Possible Sense 1	Possible Sense 2
"Safari booking"	Wildlife tour	Web browser
"Java vacation"	Java island	Java programming language
"Cruise package"	Ship journey	Cruise software/project

The system records the following user interactions:

Query	User Interaction
Amazon tour	Viewed rainforest trekking packages
Safari booking	Selected wildlife resort packages
Java vacation	Viewed hotels in Bali and Java
Cruise package	Viewed Mediterranean ship packages

However, another user searches: "**Safari download for Windows**" and clicks a browser download page.

Tasks:

1. Determine the intended sense of the ambiguous terms using the available context.
2. Identify the lexical and contextual cues that distinguish the possible senses.
3. Analyze why the same word may require different interpretations for different users.
4. Design an industrial-scale WSD strategy using query context, user history, embeddings, and click behaviour to improve travel recommendations.

4. Syntax-Driven Semantic Analysis in Legal Document Processing

A legal NLP system analyzes sentences from contracts and assigns semantic roles based on their syntactic structures.

The system produces the following analyses:

Legal Sentence	Parse Structure	Generated Semantic Interpretation
"The supplier delivered the equipment to the buyer."	Subject–Verb–Object	Supplier = Agent, Equipment = Object, Buyer = Recipient
"The buyer received the equipment from the supplier."	Subject–Verb–Object	Buyer = Agent, Equipment = Object, Supplier = Recipient
"The company terminated the contract after the violation."	Subject–Verb–Object	Company = Agent, Contract = Object
"The contract was terminated by the company."	Passive Construction	Contract = Agent, Company = Object

Tasks:

1. Analyze how syntactic structure, including active and passive constructions, influences semantic-role assignment.
2. Identify the incorrect semantic-role assignments in the given interpretations.
3. Explain how confusing grammatical subject with semantic agent can affect automated legal information extraction.
4. Recommend a syntax-driven semantic analysis architecture that can correctly handle active voice, passive voice, prepositional phrases, and long-distance dependencies in legal documents.

Answers

1. Frame Semantics in Banking Customer-Service Systems

Task 1: Semantic frame of each request

A semantic frame identifies the **main action/event** and the participants involved in that action.

Customer Query	Frame	Participants
Transfer ₹20,000 from savings account to son's account	TRANSFER	Source = savings account, Amount = ₹20,000, Destination = son's account, Customer = account holder
Block my debit card	BLOCK	Object = debit card, Customer = account holder
Send transaction statement to my email	SEND	Object = transaction statement, Destination = email, Customer = account holder
Increase daily withdrawal limit	MODIFY	Object = withdrawal limit, Customer = account holder

The relationship is:

Action → Participants/Roles

For example:

TRANSFER → Source + Amount + Destination + Customer

Task 2: Incorrect query

The incorrect interpretation is **B4**.

Actual intent:

Increase withdrawal limit

System interpretation:

Decrease withdrawal limit

The system correctly identifies the object as **WithdrawalLimit**, but incorrectly identifies the **direction of modification**.

The important semantic cue is the word "**increase**". It means the value should become larger, whereas "**decrease**" means the value should become smaller.

So:

B4

Actual: MODIFY(WithdrawalLimit, Increase, Customer)

System: MODIFY(WithdrawalLimit, Decrease, Customer)

Task 3: Effect of incorrect semantic roles

Incorrect semantic-role identification can cause serious banking errors.

For example:

Customer request:

Increase withdrawal limit

Incorrect interpretation:
Decrease withdrawal limit

If the system executes the incorrect interpretation, the customer's withdrawal limit could be reduced instead of increased.

Similarly, confusing:

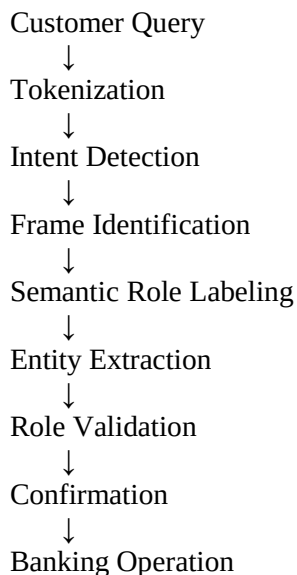
Source Account ↔ Destination Account

could potentially cause money to be transferred from the wrong account.

Therefore, banking systems should **validate important semantic roles before executing financial operations.**

Task 4: Recommended frame-based approach

A reliable banking system should use the following architecture:



For example:

"Transfer ₹20,000 from savings to son's account"

Intent = TRANSFER
Amount = ₹20,000
Source = Savings Account
Destination = Son's Account
Customer = User

The system should also use **confirmation for high-risk operations.**

Conclusion

Frame semantics improves banking chatbots by explicitly representing the **action and its participants**. This reduces ambiguity and helps prevent incorrect financial operations.

Code:

CASE STUDY 1: FRAME SEMANTICS IN BANKING

```
queries = [  
    "Transfer 20000 from my savings account to my son's account.",  
    "Block my debit card immediately.",  
    "Send my transaction statement to my email.",  
    "Increase my daily withdrawal limit."  
]  
  
frames = [  
    "TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)",  
    "BLOCK(DebitCard, Customer)",  
    "SEND(TransactionStatement, Email, Customer)",  
    "MODIFY(WithdrawalLimit, Customer)"  
]  
  
print("=" * 70)  
print("BANKING SEMANTIC FRAME ANALYSIS")  
print("=" * 70)  
  
for i in range(len(queries)):  
    print("\nQuery:", queries[i])
```

```
print("Semantic Frame:", frames[i])
```

```
# -----
```

```
# Transaction Log Analysis
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("TRANSACTION LOG ANALYSIS")
```

```
print("=" * 70)
```

```
logs = {
```

```
    "B1": ["Transfer money to another account",  
          "Transfer money"],
```

```
    "B2": ["Block debit card",  
          "Block debit card"],
```

```
    "B3": ["Receive transaction statement by email",  
          "Send statement to email"],
```

```
    "B4": ["Increase withdrawal limit",  
          "Decrease withdrawal limit"]
```

```
}
```

```
for query_id, values in logs.items():
```

```
    actual = values[0]
```

```
    system = values[1]
```

```
    print("\n", query_id)
```

```
    print("Actual Intent :", actual)
```

```
    print("System      :", system)
```

```
    if actual != system:
```

```
        print("Status      : INCORRECT")
```

```
    else:
```

```
        print("Status      : CORRECT")
```

```
# -----
```

```
# Error Analysis
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("ERROR ANALYSIS")
```

```
print("=" * 70)
```

```
print("Incorrect Query: B4")
```

```
print("Actual Intent: Increase withdrawal limit")
```

```
print("System Intent: Decrease withdrawal limit")
```

```
print("\nReason:")
```

```
print("The system misunderstood the direction of the action.")
```

```
print("\nPossible Effect:")
```

```
print("The customer's withdrawal limit could be changed incorrectly.")
```

```
# -----
```

```
# Recommended Architecture
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("RECOMMENDED FRAME-BASED APPROACH")
```

```
print("=" * 70)
```

```
print("""
```

```
Customer Query
```

```
|
```

```
v
```

```
Intent Detection
```

```
|
```

```
v
```

```
Frame Identification
```

|

v

Semantic Role Identification

|

v

Entity Extraction

|

v

Role Validation

|

v

Banking Operation

""")

print("Result:")

print("Frame semantics helps identify the action and")

print("participants correctly and reduces banking errors.")

Output:

BANKING SEMANTIC FRAME ANALYSIS

Query: Transfer 20000 from my savings account to my son's account.
Semantic Frame: TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)

Query: Block my debit card immediately.
Semantic Frame: BLOCK(DebitCard, Customer)

Query: Send my transaction statement to my email.
Semantic Frame: SEND(TransactionStatement, Email, Customer)

Query: Increase my daily withdrawal limit.
Semantic Frame: MODIFY(WithdrawalLimit, Customer)

TRANSACTION LOG ANALYSIS

B1
Actual Intent : Transfer money to another account
System : Transfer money
Status : INCORRECT

B2
Actual Intent : Block debit card
System : Block debit card
Status : CORRECT

B3
Actual Intent : Receive transaction statement by email
System : Send statement to email
Status : INCORRECT

B4
Actual Intent : Increase withdrawal limit
System : Decrease withdrawal limit
Status : INCORRECT

ERROR ANALYSIS

Incorrect Query: B4
Actual Intent: Increase withdrawal limit
System Intent: Decrease withdrawal limit

Reason:
The system misunderstood the direction of the action.

Possible Effect:
The customer's withdrawal limit could be changed incorrectly.

RECOMMENDED FRAME-BASED APPROACH

Customer Query

|
v
Intent Detection
|
v
Frame Identification
|
v
Semantic Role Identification
|
v
Entity Extraction
|
v
Role Validation
|
v
Banking Operation

Result:
Frame semantics helps identify the action and participants correctly and reduces banking errors.

2. First-Order Predicate Calculus for Smart Agriculture

Task 1: Represent observations using predicates

We can represent the field data as follows.

F1

Dry(F1)

IrrigationAvailable(F1)

Rice(F1)

F2

Wet(F2)

IrrigationAvailable(F2)

Wheat(F2)

F3

Dry(F3)

IrrigationUnavailable(F3)

Maize(F3)

F4

Wet(F4)

IrrigationUnavailable(F4)

Rice(F4)

Task 2: Determine fields requiring irrigation

Given:

$\text{Dry}(x) \wedge \text{IrrigationAvailable}(x)$
 $\rightarrow \text{NeedsWater}(x)$

and:

$\text{NeedsWater}(x)$
 $\rightarrow \text{Irrigate}(x)$

F1

Dry(F1)

IrrigationAvailable(F1)

Therefore:

$\text{NeedsWater}(F1)$
 $\text{Irrigate}(F1)$
F2

F2 is wet:

Wet(F2)

Therefore:

$\neg \text{NeedsWater}(F2)$

So it does not require irrigation.

F3

F3 is dry:

Dry(F3)

but:

IrrigationUnavailable(F3)

Therefore:

$\neg$ Irrigate(F3)

The field may need water, but the system cannot irrigate it.

F4

F4 is wet and irrigation is unavailable.

Wet(F4)

$\rightarrow \neg$ NeedsWater(F4)

Therefore, it does not require irrigation.

Final result

Field Needs water?		Irrigate?
F1	Yes	Yes
F2	No	No
F3	Yes/dry	No — unavailable
F4	No	No

Task 3: F1 and F3

Yes, F1 and F3 should be treated differently.

Both are dry:

Dry(F1)

Dry(F3)

But:

IrrigationAvailable(F1)

IrrigationUnavailable(F3)

Therefore:

F1 $\rightarrow$ Irrigate

F3 $\rightarrow$ Do not irrigate

For F3, the system should generate an alert such as:

"Field F3 requires water, but irrigation is currently unavailable."

Task 4: Is predicate logic sufficient?

No.

Predicate logic works well when information is clear and certain, but real agricultural environments contain uncertainty.

For example:

Soil moisture = 29%

Sensor confidence = 60%

Rain probability = 80%

Basic predicate logic may only represent:

Dry(F1)

or:

Wet(F1)

It does not naturally represent degrees of uncertainty.

A better system could combine:

- Predicate logic
- Fuzzy logic
- Probability
- Sensor confidence
- Weather prediction
- Historical crop data

Conclusion

Predicate logic is useful for **clear rules**, but intelligent agricultural decision support should combine logical reasoning with **uncertainty handling**.

Code:

CASE STUDY 2: FIRST-ORDER PREDICATE CALCULUS

fields = {

```

"F1": ["Dry", "IrrigationAvailable", "Rice"],
"F2": ["Wet", "IrrigationAvailable", "Wheat"],
"F3": ["Dry", "IrrigationUnavailable", "Maize"],
"F4": ["Wet", "IrrigationUnavailable", "Rice"]
}

print("=" * 60)
print("SMART AGRICULTURE PREDICATE ANALYSIS")
print("=" * 60)

# Display predicates

print("\nFIELD PREDICATES")
print("-" * 60)

for field, conditions in fields.items():

    print(field, ":")

    for condition in conditions:

        print("  ", condition + "(" + field + ")")

# -----

# Apply rules

```

```
# -----
```

```
print("\n" + "=" * 60)
```

```
print("IRRIGATION DECISIONS")
```

```
print("=" * 60)
```

```
for field, conditions in fields.items():
```

```
    soil = conditions[0]
```

```
    irrigation = conditions[1]
```

```
    crop = conditions[2]
```

```
    if soil == "Dry" and irrigation == "IrrigationAvailable":
```

```
        print(field, "-> NeedsWater")
```

```
        print(field, "-> Irrigate")
```

```
        if crop == "Rice":
```

```
            print(field, "-> PriorityIrrigation")
```

```
    elif soil == "Dry" and irrigation == "IrrigationUnavailable":
```

```
        print(field, "-> NeedsWater")
```

```
        print(field, "-> DO NOT Irrigate")
```

```
        print(field, "-> Irrigation Unavailable")
```

```
elif soil == "Wet":
```

```
    print(field, "-> NOT NeedsWater")
```

```
    print(field, "-> DO NOT Irrigate")
```

```
# -----
```

```
# F1 and F3 comparison
```

```
# -----
```

```
print("\n" + "=" * 60)
```

```
print("F1 VS F3")
```

```
print("=" * 60)
```

```
print("F1: Dry + Irrigation Available")
```

```
print("F1: Irrigate")
```

```
print("\nF3: Dry + Irrigation Unavailable")
```

```
print("F3: Do NOT Irrigate")
```

```
print("\nResult:")
```

```
print("F1 and F3 must be treated differently.")
```

```
# -----
```

```
# Limitation
```

```
# -----
```

```
print("\n" + "=" * 60)
```

```
print("LIMITATION OF PREDICATE LOGIC")
```

```
print("=" * 60)
```

```
print("Predicate logic works with clear information.")
```

```
print("Real sensor data may be incomplete or uncertain.")
```

```
print("Therefore, probability or fuzzy logic can also be used.")
```

Output:

```
=====
SMART AGRICULTURE PREDICATE ANALYSIS
=====

FIELD PREDICATES
-----
F1 :
    Dry(F1)
    IrrigationAvailable(F1)
    Rice(F1)
F2 :
    Wet(F2)
    IrrigationAvailable(F2)
    Wheat(F2)
F3 :
    Dry(F3)
    IrrigationUnavailable(F3)
    Maize(F3)
F4 :
    Wet(F4)
    IrrigationUnavailable(F4)
    Rice(F4)
```

IRRIGATION DECISIONS

```
F1 -> NeedsWater
F1 -> Irrigate
F1 -> PriorityIrrigation
F2 -> NOT NeedsWater
F2 -> DO NOT Irrigate
F3 -> NeedsWater
F3 -> DO NOT Irrigate
F3 -> Irrigation Unavailable
F4 -> NOT NeedsWater
F4 -> DO NOT Irrigate
```

F1 VS F3

```
F1: Dry + Irrigation Available
F1: Irrigate
```

```
F3: Dry + Irrigation Unavailable
F3: Do NOT Irrigate
```

Result:
F1 and F3 must be treated differently.

LIMITATION OF PREDICATE LOGIC

Predicate logic works with clear information.
Real sensor data may be incomplete or uncertain.
Therefore, probability or fuzzy logic can also be used.

3. Word Sense Disambiguation in Travel Recommendation

Task 1: Determine intended senses

Query	Context	Intended Sense
Amazon tour	Viewed rainforest trekking packages	Amazon rainforest
Safari booking	Selected wildlife resorts	Wildlife tour
Java vacation	Viewed hotels in Bali and Java	Java island
Cruise package	Viewed Mediterranean ship packages	Ship journey
Safari download for Windows	Clicked browser download page	Safari web browser

The last example is especially important because the context completely changes the meaning.

Task 2: Lexical and contextual cues

Amazon

"Amazon tour"

"rainforest"
"trekking"

→ Amazon rainforest.

But:

Amazon website
Amazon shopping
Amazon products

→ Amazon company.

Safari

Safari booking
wildlife
resort
animals

→ Wildlife safari.

But:

Safari download
Windows
browser

→ Safari web browser.

Java

Java vacation
hotels
Bali
travel

→ Java island.

But:

Java programming
code
developer
software

→ Java programming language.

Cruise

Mediterranean
ship
package
journey

→ Ship cruise.

But software/project-related terms could indicate a different sense.

Task 3: Why does the same word have different meanings?

A word can have multiple senses depending on:

- Surrounding words
- User history
- Search context
- Previous clicks
- Location
- User interests
- Search intent

For example:

Safari booking

suggests a wildlife trip.

But:

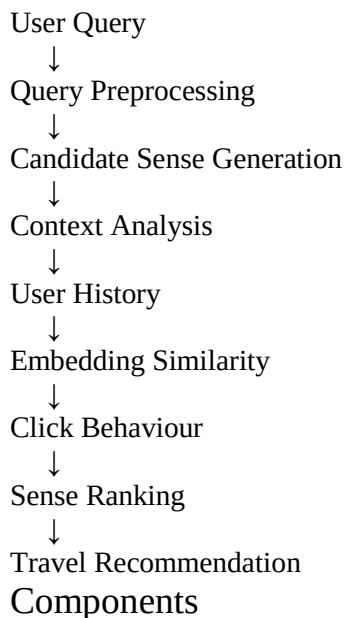
Safari download for Windows

suggests a browser.

Therefore, WSD should not depend only on the ambiguous word itself.

Task 4: Industrial-scale WSD strategy

A practical architecture would be:



1. Query context

Use surrounding words such as:

tour
rainforest
booking
download
Windows

2. User history

Previous searches and clicks can indicate the user's interests.

3. Embeddings

Represent query and possible meanings as vectors and calculate semantic similarity.

4. Click behaviour

If users repeatedly click wildlife packages after searching "Safari", increase the probability of the wildlife sense.

5. Continuous learning

The system can learn from new searches and clicks.

Conclusion

Industrial WSD should combine **query context + embeddings + user history + click behaviour** rather than using only dictionary definitions.

Code:

CASE STUDY 3: WORD SENSE DISAMBIGUATION

queries = {

 "Amazon tour":

 ["Amazon rainforest", "Amazon company"],

 "Safari booking":

 ["Wildlife tour", "Web browser"],

"Java vacation":

["Java island", "Java programming language"],

"Cruise package":

["Ship journey", "Cruise software"],

"Safari download for Windows":

["Wildlife tour", "Web browser"]

}

Context from user behaviour

context = {

"Amazon tour":

"Viewed rainforest trekking packages",

"Safari booking":

"Selected wildlife resort packages",

"Java vacation":

"Viewed hotels in Bali and Java",

"Cruise package":

"Viewed Mediterranean ship packages",

"Safari download for Windows":

"Clicked a browser download page"

}

print("=" * 70)

print("WORD SENSE DISAMBIGUATION")

print("=" * 70)

for query in queries:

print("\nQuery:", query)

print("Context:", context[query])

Simple context rules

text = (query + " " + context[query]).lower()

if "rainforest" in text or "trekking" in text:

sense = "Amazon rainforest"

elif "wildlife" in text or "resort" in text:

sense = "Wildlife tour"

elif "java" in query.lower() and "hotel" in text:

```
sense = "Java island"
```

```
elif "mediterranean" in text or "ship" in text:
```

```
sense = "Ship journey"
```

```
elif "download" in text or "windows" in text or "browser" in text:
```

```
sense = "Web browser"
```

```
else:
```

```
sense = "Unknown"
```

```
print("Selected Sense:", sense)
```

```
# -----
```

```
# WSD Strategy
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("INDUSTRIAL-SCALE WSD STRATEGY")
```

```
print("=" * 70)
```

```
print("""
```

```
User Query
```

```
|
```

v

Query Context

|

v

Candidate Senses

|

v

User History

|

v

Embeddings

|

v

Click Behaviour

|

v

Sense Ranking

|

v

Travel Recommendation

""")

print("Result:")

print("The same word can have different meanings depending")

print("on context and user behaviour.")

Output:

=====

WORD SENSE DISAMBIGUATION

=====

Query: Amazon tour
Context: Viewed rainforest trekking packages
Selected Sense: Amazon rainforest

Query: Safari booking
Context: Selected wildlife resort packages
Selected Sense: Wildlife tour

Query: Java vacation
Context: Viewed hotels in Bali and Java
Selected Sense: Java island

Query: Cruise package
Context: Viewed Mediterranean ship packages
Selected Sense: Ship journey

Query: Safari download for Windows
Context: Clicked a browser download page
Selected Sense: Web browser

=====

INDUSTRIAL-SCALE WSD STRATEGY

=====

```
graph TD
    UserQuery[User Query] --> V1((v))
    V1 --> QueryContext[Query Context]
    QueryContext --> V2((v))
    V2 --> CandidateSenses[Candidate Senses]
    CandidateSenses --> V3((v))
    V3 --> UserHistory[User History]
    UserHistory --> V4((v))
    V4 --> Embeddings[Embeddings]
    Embeddings --> V5((v))
    V5 --> ClickBehaviour[Click Behaviour]
    ClickBehaviour --> V6((v))
    V6 --> SenseRanking[Sense Ranking]
    SenseRanking --> V7((v))
    V7 --> TravelRecommendation[Travel Recommendation]
```

Result:

The same word can have different meanings depending on context and user behaviour.

4. Syntax-Driven Semantic Analysis in Legal Documents

Task 1: Syntax and semantic roles

Syntax determines how words are grammatically connected, but grammatical position does **not always equal semantic role**.

Active voice

The supplier delivered the equipment to the buyer.

Correct roles:

Supplier = Agent

Equipment = Object/Theme

Buyer = Recipient

Second sentence

The buyer received the equipment from the supplier.

Correct roles:

Buyer = Recipient

Equipment = Object/Theme

Supplier = Source

Although **buyer** is the grammatical subject, it is not the Agent of the delivery event.

Passive voice

The contract was terminated by the company.

Correct roles:

Contract = Object/Theme

Company = Agent

The grammatical subject is **contract**, but the semantic Agent is **company**.

Task 2: Incorrect semantic-role assignments

There are **two major incorrect interpretations**.

Sentence 2

Given:

Buyer = Agent

Supplier = Recipient

This is incorrect.

Correct:

Buyer = Recipient
Supplier = Source/Agent
Sentence 4

Given:

Contract = Agent
Company = Object

This is incorrect.

Correct:

Contract = Object/Theme
Company = Agent
Sentence 1

This is essentially correct:

Supplier = Agent
Equipment = Object
Buyer = Recipient
Sentence 3

Also correct:

Company = Agent
Contract = Object

Task 3: Why confusing subject and Agent is dangerous

A grammatical subject is not necessarily the semantic Agent.

For example:

The contract was terminated by the company.

Grammatical subject:

contract

Semantic Agent:

company

If a legal NLP system assumes:

Subject = Agent

it could incorrectly conclude:

The contract terminated the company.

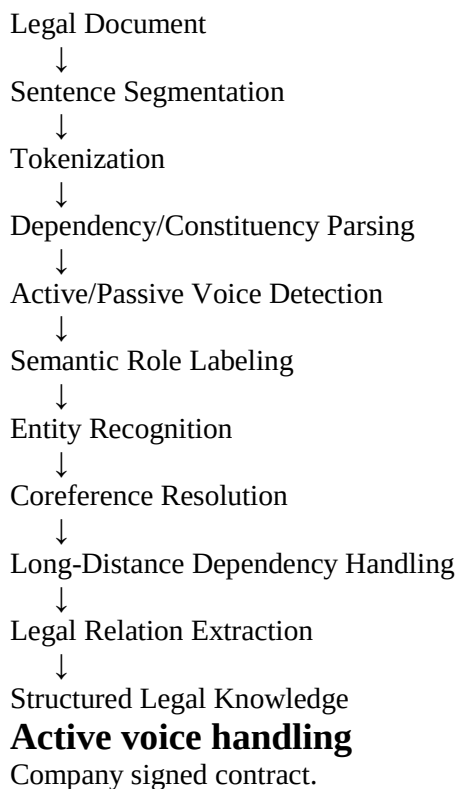
This can produce serious errors in:

- Contract analysis
- Liability identification
- Legal information extraction
- Compliance systems
- Case analysis
- Obligation extraction

Therefore, legal NLP needs **semantic role labeling in addition to syntactic parsing**.

Task 4: Recommended architecture

A strong legal NLP system can use:



Company → Agent
Contract → Object

Passive voice handling
Contract was signed by company.

Contract → Object
Company → Agent

Prepositional phrase handling
Supplier delivered equipment to buyer.

Supplier → Agent
Equipment → Object
Buyer → Recipient

Long-distance dependency

The system should use dependency parsing and coreference resolution to connect participants even when they are separated by long clauses.

Conclusion

A syntax-driven semantic system should combine **dependency parsing, voice detection, semantic role labeling, coreference resolution, and legal-domain rules**. This prevents the common error of assuming that the grammatical subject is always the semantic Agent.

Code:

LEGAL SEMANTIC ROLE ANALYSIS

Sentence:

The supplier delivered the equipment to the buyer.

Semantic Roles:

Supplier -> Agent

Equipment -> Object

Buyer -> Recipient

Sentence:

The buyer received the equipment from the supplier.

Semantic Roles:

Buyer -> Recipient

Equipment -> Object

Supplier -> Agent/Source

Sentence:

The company terminated the contract after the violation.

Semantic Roles:

Company -> Agent

Contract -> Object

Sentence:

The contract was terminated by the company.

Semantic Roles:

Contract -> Object

Company -> Agent

ERROR ANALYSIS

Sentence 2:

Incorrect: Buyer = Agent

Correct: Buyer = Recipient

Correct: Supplier = Agent/Source

Sentence 4:

Incorrect: Contract = Agent

Incorrect: Company = Object

The company terminated the contract.

Company -> Agent
Contract -> Object

Passive:

The contract was terminated by the company.
Contract -> Object
Company -> Agent

RECOMMENDED LEGAL NLP ARCHITECTURE

```
Legal Document
|
v
Tokenization
|
v
Syntactic Parsing
|
v
Active/Passive Detection
|
v
Semantic Role Labeling
|
v
Entity Recognition
|
v
Coreference Resolution
|
v
Legal Relation Extraction
```

Result:

Semantic analysis should not assume that
the grammatical subject is always the Agent.
Syntax and semantic roles must be analyzed together.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply

		to interpret data accurately			concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Understanding of Data	Analysis	Application of Concepts	Conclusion	Clarity	Sub. Total	Total %
1	3	4	4	3	3	20	85
2	4	4	3	3	3	20	
3	3	3	4	4	3	25	
4	3	3	3	4	4	20	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____